

Dataset

Dictionaries in, dictionaries out

Gregory S. DeLozier, PhD

Two Answers to One Question

	Peewee	Dataset
You start with	Model classes	A database, or a dictionary
Rows are	Objects	Dictionaries
The schema comes from	The models	The database, or the first rows
Suits	Applications with rules	Exploration and scratch work

How much structure should the code impose?

Installing Dataset

```
python3 -m venv .venv  
source .venv/bin/activate  
python3 -m pip install -r requirements.txt  
python3 -m pip show dataset
```

```
dataset==2.0.0  
Flask==3.1.3  
tzdata==2026.4
```

Python **3.10** or newer.

A Database You Did Not Design

```
import dataset

db = dataset.connect("sqlite:///pets.db")
print(db.tables)
print(db["pets"].columns)
print(len(db["pets"]))
```

```
['kind', 'pets']
['id', 'name', 'age', 'kind_id', 'owner']
4
```

No model classes.

Rows Are Dictionaries

```
suzy = db["pets"].find_one(name="Suzy")  
print(suzy["age"])
```

```
OrderedDict({'id': 1, 'name': 'Suzy', 'age': 3,  
            'kind_id': 1, 'owner': 'Greg'})
```

`dict(row)`, `row.get("age")`, JSON.

Asking Questions

```
pets.find(kind_id=[1, 3])
pets.find(age={">=": 3}, order_by="-age", _limit=2)
pets.count(kind_id=1)
pets.distinct("owner")
```

Operator	Meaning
> >= < <= !=	Comparison
in not in between	Sets and ranges
like ilike startswith endswith	Text

When the API Runs Out, Write SQL

```
for row in db.query(
    "select kind_id, count(*) as n "
    "from pets group by kind_id"):
    print(dict(row))
```

```
{'kind_id': 1, 'n': 2}
{'kind_id': 2, 'n': 1}
{'kind_id': 3, 'n': 1}
```

Postponed, not hidden.

Changing Data

```
pets.insert({"name": "Buddy", "age": 5, ...})
pets.insert_many([...])
pets.update({"name": "Buddy", "age": 6}, ["name"])
pets.upsert({"name": "Buddy", "age": 7}, ["name"])
pets.delete(name="Tom")
```

Call	Returns
insert	New row identifier
update	Rows changed
upsert	True when updated; new identifier when inserted
delete	True if a row went

A Transaction Is a `with` Block

```
with dataset.connect("sqlite:///pets.db") as tx:  
    tx["pets"].insert({"name": "Inside", ...})  
    tx["pets"].insert({"name": "Bad", "age": -1, ...})
```

The second insert fails: **neither row is stored.**

Tables That Make Themselves

```
scratch = db["scratch"]
scratch.insert({"a": 1, "b": 2.5, "c": True, "d": "hi"})
```

Python value	Column type
int	BIGINT
float	FLOAT
bool	BOOLEAN
date, datetime	DATE, timestamp
dict	JSON
anything else	TEXT

Plus an integer id.

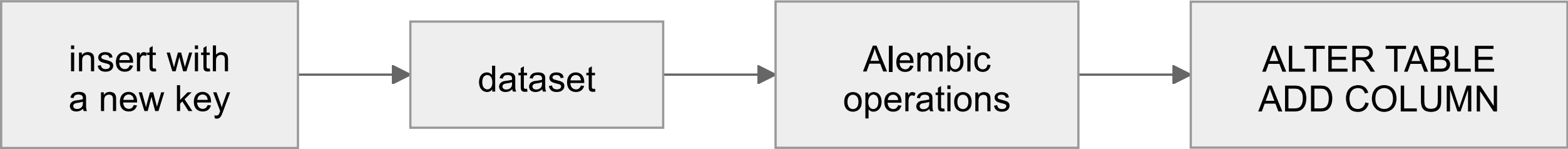
A New Key Adds a Column

```
print(pets.columns)
pets.insert(..., "breed": "Beagle")
print(pets.columns)
```

```
['id', 'name', 'age', 'kind_id', 'owner']
['id', 'name', 'age', 'kind_id', 'owner', 'breed']
```

Old rows: breed is NULL. A typo such as aeg becomes a column too.

What Alembic Does



Full Alembic	What dataset uses
Versioned migration scripts	No
alembic_version table	No
Upgrade and downgrade	No
Operation that adds a column	Yes

What Dataset Leaves to You

	Who supplies it
NOT NULL, CHECK	The SQL script; SQLite enforces
Rules never written	Nobody
Foreign keys	You must turn them on
Joins	You, in SQL
Write-ahead logging	Dataset, by default

Foreign Keys Are Off Until Asked

```
db = dataset.connect(  
    "sqlite:///pets.db",  
    on_connect_statements=["PRAGMA foreign_keys=ON"])
```

Without it: a pet with kind_id 999 goes in, and a kind in use can be deleted.

The Pets Application with Dataset

Layer	Where
Browser	<code>required, type="number" min="0"</code>
Route	<code>check_pet_form</code>
Table	<code>NOT NULL, CHECK, foreign key in create_db.sql</code>

One join query lists the pets. Kind delete: the database refuses; the route reports it.

Exploring mystery.db

1. Tables and views
2. Columns and row counts
3. A few rows
4. A join to test a guess
5. The views
6. Interesting values
7. The unrelated table
8. Keep what you found

```
db = dataset.connect(  
    "sqlite:///file:mystery.db?mode=ro&uri=true",  
    sqlite_wal_mode=False)
```

Read-Only Means Read-Only

Setting	Effect
mode=ro	Every write refused
sqlite_wal_mode=False	No switch to write-ahead logging

A file already switched stays switched. Explore a **copy**.

What Did We Find?

```
{'script': 'CJK', 'characters': 28209, ...}  
{'script': 'HANGUL', 'characters': 11625, ...}
```

But the first character row said `'script': 'SPACE'`.

The `script` column is the first word of the name.

Choosing a Tool

	Direct SQL	Peewee	Dataset
You write	SQL	Models	Dictionaries
Rules declared in	SQL	Models	SQL, outside
Changing a table	By hand	Manual or tool	Automatic, unreviewed
Suits	Full control	Applications	Exploration

Exercise: Explore Something Else

Pick an SQLite database you did not create.

Write down each guess before you test it.

Which guesses were wrong?

Save one result to a CSV file.

Where the Rules Live

Dictionaries: **explore quickly.**

SQL: **declare the rules.**

Foreign keys: **turn them on.**